

CS305 Programming Languages Project Report

Zeynep Eda Erkoçak 240302222



**INTERNATIONAL
UNIVERSITY OF SARAJEVO**

Introduction and Project Concept

Language Name: MeowLang

Theme: Cat Behavior and Chaotic Kitten Concept

For this project, I made a mini programming language called MeowLang. My main goal was to make a fun, dramatic and creative language inspired by cats. In normal programming languages keywords are very boring. In MeowLang, I changed all keywords to describe cat behaviors. For example, variables are small "kittens", loops are fast "zoomies" and if-else conditions are driven by suspicious "sniffs" or angry "hisses".

This way the language feeld more playful and unique, while still showing the main of programming.

Syntax and Keywords Mapping

MeowLang Keyword	Standard Programming Meaning	Description
kitten	Var/int/declaration	Used when you want to make a new variable
=	=/assignment	Assigns a number or calculation to a variable
meow	Print()/output	Prints a variable value or number to the screen
sniff	if/selection	Starts a check to see if something is true
hiss	Else/aternative	Runs when the sniff condition is false
zoomies	While/iteration	Starts a loop block that runs until the condition ends
sleep	End/closing bracket	Closes the body of a loop or condition block
text	String	Used to represent and print plain test messages enclosed in double quotes

Formal Specifications & Rules

MeowLang support mathematical operations and comparison rules:

- **Arithmetic Operations:** It supports addition (+), subtraction (-), multiplication (*), division (/) and modulo (%). My interpreter uses floor division (//) for division because it keeps numbers as integers.
- **Comparison Operators:** It uses standard symbols like ==, !=, >, <, >= and <=.

- **Block System:** Every sniff (if) and zoomies (while) block must finish with the sleep keyword. This helps the code understand where the block ends.

Implementation Details

In this section, I will explain how my Python code implements the compiler/interpreter components for MeowLang

A)The Lexical Analyzer (lexer.py):

The Lexer reads the MeowLang code and splits it into small pieces called tokens. It uses Regex to find and match the special cat keywords we made.

```
# grammar and keyword rules for MeowLang
self.spec = [
    (r'\s+', None), # skips spaces and new lines
    (r'kitten\b', 'VARIABLE'),
    (r'meow\b', 'PRINT'),
    (r'zoomies\b', 'WHILE'),
    (r'sleep\b', 'END'),
    (r'sniff\b', 'IF'),
    (r'hiss\b', 'ELSE'),
    (r'"[^"\n]*"', 'STRING'), # rule for matching string literals
    (r'[0-9]+', 'NUMBER'),
    (r'[a-zA-Z_][a-zA-Z0-9_]*', 'IDENTIFIER'),
    (r'==', 'OPERATOR'),
    (r'>=', 'OPERATOR'),
    (r'<=', 'OPERATOR'),
    (r'!=', 'OPERATOR'),
    (r'=', 'OPERATOR'),
    (r'\+', 'OPERATOR'),
    (r'-', 'OPERATOR'),
    (r'\*', 'OPERATOR'),
    (r '/', 'OPERATOR'),
    (r'%', 'OPERATOR'),
    (r'>', 'OPERATOR'),
    (r'<', 'OPERATOR'),
]
```

In this part of the code I defined all the token rules. For example kitten\b creates a VARIABLE token and meow\b creates a PRINT token. The lexer uses a while loop to scan characters one by one. If it sees a space it skips it. If it sees a newline, it updates the line number. If a character does not match any rule it throws a SyntaxError.

B) The Parser (parser.py):

The Parser reads the list of tokens from the Lexer and builds the Abstract Syntax Tree (AST) using nodes from astNodes.py.

```
# handles variable declaration (kitten keyword)
if token.type == "VARIABLE":
    name = self.tokens[self.position + 1].value
    # check if it is an inline math assignments (exp. x=a*b)
    if self.position + 4 < len(self.tokens) and
self.tokens[self.position + 4].value in ["+", "-", "*", "/", "%"]:
        left = self.tokens[self.position + 3].value
        operator = self.tokens[self.position + 4].value
        right = self.tokens[self.position + 5].value
        nodes.append(AssignNode(name, left, operator, right))
        self.position += 6
    else:
        value = self.tokens[self.position + 3].value
        nodes.append(VariableNode(name, value))
        self.position += 4
```

The parser goes through the tokens one by one. When it finds a VARIABLE token (kitten), it looks at what comes next. If it sees an assignment sign and math symbols like + or -, it makes an AssignNode. If not it just makes a VariableNode. For loops and conditions it keeps reading statements until it reaches the END (sleep) token.

C) The Execution Engine / Interpreter (interpreter.py):

The Interpreter is the part that runs the program. It follows the code rules and keeps track of the variables in memory.

```
def _get_value(self, val):
    # helper to find if the value is int or a variable lookup
    if isinstance(val, int):
        return val
    if isinstance(val, str):
        # if it is a string literal inside quotes, strip them and return
the text
        if val.startswith('"') and val.endswith('"'):
            return val[1:-1]
        if val.isdigit():
            return int(val)

    if val not in self.variables:
        MeowError.lost_toy_error("Runtime", val)

    return self.variables.get(val, 0)
```

```

        return 0

    def execute_body(self, body_nodes):
        # runs internal blocks for conditions and loops (if/while statements)
        for cmd in body_nodes:
            if isinstance(cmd, PrintNode):
                print(self._get_value(cmd.value))
            elif isinstance(cmd, AssignNode):
                left_val = self._get_value(cmd.left)
                right_val = self._get_value(cmd.right)

                res = 0
                if cmd.operator == "+": res = left_val + right_val
                elif cmd.operator == "-": res = left_val - right_val
                elif cmd.operator == "*": res = left_val * right_val
                elif cmd.operator == "/":
                    if right_val == 0: MeowError.empty_bowl_error("Runtime")
                    res = left_val // right_val
                elif cmd.operator == "%":
                    if right_val == 0: MeowError.empty_bowl_error("Runtime")
                    res = left_val % right_val
                self.variables[cmd.name] = res

```

I made a Python dictionary called `self.variables` to act like memory for the program. When a new variable is created it gets saved inside this dictionary. The `execute_body` method runs the commands that are inside loops or conditions. For math, it can do the basic five operations. For division it uses integer division (`//`) so the results stay clean and it avoids problems like dividing by zero.

```

# print statement(meow)
    elif isinstance(node, PrintNode):
        print(self._get_value(node.value))
    # math assignments(exp. x=y+1)
    elif isinstance(node, AssignNode):
        left_val = self._get_value(node.left)
        right_val = self._get_value(node.right)

        result = 0
        if node.operator == "+": result = left_val + right_val
        elif node.operator == "-": result = left_val - right_val
        elif node.operator == "*": result = left_val * right_val
        elif node.operator == "/": result = left_val // right_val if
right_val != 0 else 0
        elif node.operator == "%": result = left_val % right_val if
right_val != 0 else 0
        self.variables[node.name] = result

```

```

        # selection statement (sniff-hiss)
        elif isinstance(node, IfNode):
            left_val = self.variables.get(node.variable, 0)
            right_val = self._get_value(node.value)

            if self.check_condition(left_val, node.operator, right_val):
                self.execute_body(node.body)

```

This part shows how control statements work. When the interpreter sees an IfNode (sniff) it checks the condition with check_condition. If the result is false, it goes to the else part (hiss). For a WhileNode (zoomies), it keeps running with while True and checks the condition again and again. When the condition fails it stops safely with a break.

D)Error Handling Mechanism (error.py)

The Error Handler is the part that protects the language from crashes. It intercepts runtime bugs and prints creative, cat-themed alerts instead of generic Python failures.

```

@staticmethod
def hairball_error(line, details=""):
    # Handles syntax and dynamic token errors
    print(f"🐾 [HAIRBALL ERROR ON LINE {line}]: Cat choked on syntax!
{details}")
    sys.exit(1)

@staticmethod
def lost_toy_error(line, name):
    # When an undeclared variable is accessed
    print(f"🐱 [LOST TOY ERROR ON LINE {line}]: Kitten lost the variable
'{name}' under the sofa/scope!")
    sys.exit(1)

@staticmethod
def scratch_error(line, details=""):
    # Handles invalid runtime or math logical structures
    print(f"😾 [SCRATCH ERROR ON LINE {line}]: Angry cat scratched the
system! {details}")
    sys.exit(1)

@staticmethod
def empty_bowl_error(line):
    # Prevents divisions or modulus by zero
    print(f"🍲 [EMPTY BOWL ERROR ON LINE {line}]: Empty bowl! You cannot
divide treats by zero portions!")
    sys.exit(1)

```

Example Programs and System Outputs

I tested my interpreter using three different test scripts to prove that all required structures work perfectly (variables, 5 arithmetic operations, selection and iteration)

Example 1:

```
kitten kibble_count=3
kitten danger_limit=5
sniff kibble_count<danger_limit
    meow "Kitten is hungry, give more fish!"
hiss
    meow "Kitten is happy and full!"
sleep
```

This program tries out the sniff-hiss selection and the new string feature. It checks if the cat is hungry and prints a message. The lexer breaks down the condition into tokens, and the parser turns it into an IfNode. When the program runs, the interpreter checks the condition. If it's true, it skips the hiss (else) part and goes straight to the statement inside sniff.

Execution Output:



```
Drive\Masaüstü\programming10Deneme2\main.py"
TOKENS:
VARIABLE:kitten
IDENTIFIER:kibble_count
OPERATOR:=
NUMBER:3
VARIABLE:kitten
IDENTIFIER:danger_limit
OPERATOR:=
NUMBER:5
IF:sniff
IDENTIFIER:kibble_count
OPERATOR:<
IDENTIFIER:danger_limit
PRINT:meow
STRING:"Kitten is hungry, give more fish!"
ELSE:hiss
PRINT:meow
STRING:"Kitten is happy and full!"
END:sleep

OUTPUT:
Kitten is hungry, give more fish!
PS C:\Users\zynpe\OneDrive\Masaüstü\programming10Deneme2>
```

Example 2:

```
kitten current_xp = 8
kitten level_limit = 5
kitten base_treats = 6
sniff current_xp>level_limit

    kitten total_treats=base_treats*2

    kitten final_portions=total_treats/3

    meow "Calculated final treat portions:"

    meow final_portions
hiss

    meow base_treats
sleep
```

This program checks math inside a sniff block. It uses multiplication and integer division to figure out treat portions. It shows that the interpreter can handle variables that change inside the block, follow the math order step by step, and keep local variables safe in memory without losing them while running.

Execution Output:

```
TOKENS:
VARIABLE:kitten
IDENTIFIER:current_xp
OPERATOR:=
NUMBER:8
VARIABLE:kitten
IDENTIFIER:level_limit
OPERATOR:=
NUMBER:5
VARIABLE:kitten
IDENTIFIER:base_treats
OPERATOR:=
NUMBER:6
IF:sniff
IDENTIFIER:current_xp
OPERATOR:>
IDENTIFIER:level_limit
VARIABLE:kitten
IDENTIFIER:total_treats
OPERATOR:=
IDENTIFIER:base_treats
OPERATOR:*
NUMBER:2
VARIABLE:kitten
IDENTIFIER:final_portions
OPERATOR:=
IDENTIFIER:total_treats
OPERATOR:/
NUMBER:3
PRINT:meow
STRING:"Calculated final treat portions:"
PRINT:meow
IDENTIFIER:final_portions
ELSE:hiss
PRINT:meow
IDENTIFIER:base_treats
END:sleep

OUTPUT:
Calculated final treat portions:
4
```


Example 3:

```
kitten energy = 7
kitten nap_code = 4444
zoomies energy > 1
    meow "Current kitten energy level:"
    meow energy
    energy = energy - 2
sleep
meow "Kitten is completely exhausted!"
meow nap_code
```

This program checks the zoomies loop by making a countdown. It lowers the kitten's energy step by step until it goes to sleep. The interpreter shows that the loop works correctly. On each round, it updates the variable in memory, keeps track of the changes, and stops safely when the condition is no longer true.

Execution Output:

```
TOKENS:
VARIABLE:kitten
IDENTIFIER:energy
OPERATOR:=
NUMBER:7
VARIABLE:kitten
IDENTIFIER:nap_code
OPERATOR:=
NUMBER:4444
WHILE:zoomies
IDENTIFIER:energy
OPERATOR:>
NUMBER:1
PRINT:meow
STRING:"Current kitten energy level:"
PRINT:meow
IDENTIFIER:energy
IDENTIFIER:energy
OPERATOR:=
IDENTIFIER:energy
OPERATOR:-
NUMBER:2
END:sleep
PRINT:meow
STRING:"Kitten is completely exhausted!"
PRINT:meow
IDENTIFIER:nap_code

OUTPUT:
Current kitten energy level:
7
Current kitten energy level:
5
Current kitten energy level:
3
Kitten is completely exhausted!
4444
PS C:\Users\zynpe\OneDrive\Masaüstü\programming10Deneme2>
```

Integration and Execution Architecture

The main.py file is like the main controller of MeowLang. It reads the source code, sends it to the lexer to split into tokens, gives those tokens to the parser to build the tree and then starts the interpreter to run the program.

```
# main file to execute everything
from lexer import Lexer
from parser import Parser
from interpreter import Interpreter

with open("program.meow", "r", encoding="utf-8") as f:
    code = f.read()

#step 1:run the lexer to go get tokens
lexer = Lexer(code)
tokens = lexer.tokenize()
print("TOKENS:")
for t in tokens:
    print(t)

#step 2:run the parser to get AST nodes
parser = Parser(tokens)
nodes = parser.parse()

#step3: pass nodes into the interpreter
interpreter = Interpreter()
print("\nOUTPUT:")
interpreter.run(nodes)
```

Testing Errors

To keep the program safe, MeowLang has its own error system in error.py. This system catches problems during compile time and runtime, so the program doesn't just crash like normal Python. Below are some examples that show how errors are handled.

A)Zero Division Trap Error

Testing an active division or modulo operation where the divisor evaluates to zero.

For example:

```
kitten treats=10
```

```
kitten zero_box=0
```

```
kitten result=treats/zero_box
```

When we enter this code into the program, the following output will appear;

OUTPUT:

```
🐛 [EMPTY BOWL ERROR ON LINE Runtime]: Empty bowl, You cannot divide treats by zero portions!  
PS C:\Users\zynpe\OneDrive\Masaüstü\programming10Deneme2> █
```

B) Name Not Found Error

This test shows what happens when the program tries to use a variable that was never declared or is outside its scope. In that case, the interpreter cannot find the name and gives an error.

For example:

```
kitten health=100
```

```
meow damage_points
```

This will be outputted;

OUTPUT:

```
🐛 [LOST TOY ERROR ON LINE Runtime]: Kitten lost the variable 'damage_points' under the sofa/scope  
PS C:\Users\zynpe\OneDrive\Masaüstü\programming10Deneme2> █
```

Conclusion

In conclusion, MeowLang shows that even a playful language can teach real programming ideas. It uses cat-style words for normal programming parts like variables, loops, if-else and math. This makes the language fun but also clear for learning.

The project proves that the pipeline works step by step:

- The lexer can split the code into tokens.
- The parser builds the tree structure.
- The interpreter runs the program, keeps track of memory, and handles errors with its own system.

Through the examples MeowLang showed it can manage variables, do math, check conditions and run loops safely. It also catches mistakes instead of crashing, which makes it more reliable.

Overall, MeowLang is a small but complete project. It connects theory with practice, shows how compilers and interpreters work and gives students a creative way to understand programming language design. (at least it was useful for me 😊)